AI Service Navigator – frontend task split

Who builds what, in what order, and how we merge · InclusivAI Hackathon, CAIRO.thws · Thu 29 Oct 2026 · based on the Frontend Handoff Spec v0.1

> **Golden rule: every file has exactly one owner.** Never edit the other person's files; change things through `CONTRACT.md`. B's `js/ui.js` is the only code that touches the page. A's `app.js` imports `ui.js` and calls its functions; `ui.js` never imports `app.js`, it only calls the handlers it was given. Plain HTML, CSS and JS: no build step, no CDN, no web fonts (spec §9).

1 · Who builds what

A · ENGINE

Claude Code + design canvas · state, audio, backend

Owns: `app.js`, `js/api.js`, `js/recorder.js`, `js/player.js`, `js/mock.js`, `AGENTS.md`, `CONTRACT.md`, `SPEC.md`, `README.md`

- ☐ **Skeleton** – repo, `SPEC.md` (this spec as text), `AGENTS.md`, `CLAUDE.md`, `GEMINI.md`, `CONTRACT.md`, empty files for both sides by 11:30
- ☐ **Design canvas** – 5 talk-button states, conversation, service card, 360 px, dark mode; neutral colours, system fonts. Send B screenshots plus colour and size values by 12:00
- ☐ `js/mock.js` – `?mock=1` returns the six demo scenarios; `?mock=1&error=<code>` simulates each error §11
- ☐ `js/recorder.js` – getUserMedia; MediaRecorder `audio/webm;codecs=opus`, default type as Safari fallback; 30 s cap; level meter; always stop all tracks §9
- ☐ `js/player.js` – play, stop and replay reply audio (`API_BASE` + `reply_audio_url`) §6
- ☐ `js/api.js` – `API_BASE` (default localhost:8000); `POST /talk` multipart: audio or text, language, session_id; `GET /health` on load; 60 s timeout; map errors to codes §6
- ☐ `app.js` – state machine idle → listening → thinking → speaking / error; hold vs tap (≈400 ms, ignore the trailing click); barge-in; Space toggles, Esc stops playback; new session_id on Start over §5
- ☐ **Real backend** – switch mock off, test Chrome and Safari, check multi-turn context 13:00–14:00
- ☐ **README** – how to run: `python3 -m http.server`, backend URL, `?mock=1` §11

Done when acceptance criteria 1, 4 and 5 pass, first with `?mock=1`, then with the real backend.

B · INTERFACE

Gemini · page, look, text, accessibility

Owns: `index.html`, `styles.css`, `js/i18n.js`, `js/ui.js`

- ☐ `index.html` – header (name, DE/EN toggle, A− / A+), notice banner, conversation list, talk button, text input + send, Start over. Semantic HTML; no visible text hard-coded §4
- ☐ `js/i18n.js` – one object with `de` and `en` keys for every string, incl. all 8 error messages; default from navigator.language (German if unknown); the toggle wins and is saved in localStorage inside try/catch; set `<html lang>` §7
- ☐ `js/ui.js` – exactly the functions in `CONTRACT.md`; append-only aria-live conversation list; replay button on assistant bubbles; service card as a labelled region with a heading; unknown extra fields shown as simple rows §4 §8
- ☐ `styles.css` – neutral theme from A's design; 18 px base; A− / A+ up to 200 %; 48 px targets; 96 px talk button with icon + label + colour per state (never colour alone); light and dark with an explicit body background; reduced motion; no horizontal scroll at 360 px; inline SVG icons §8
- ☐ **Accessibility pass** – keyboard only; screen reader (VoiceOver or NVDA); visible focus ring; aria-pressed and a state-based aria-label on the talk button §8
- ☐ **Copy check** – short plain sentences (A2–B1), no jargon, both languages complete §7

Done when acceptance criteria 2, 3, 6, 7 and 8 pass.

Rules for both AIs (put these in AGENTS.md and in every prompt)

- Edit only your own files. If you need a change in the other person's file, ask; don't let your AI touch it.
- Small commits that say what works. Merge `main` into your branch before every new AI task.
- Serve the page over localhost (`python3 -m http.server 5500`): split JS modules don't load from file://, and the microphone needs localhost or HTTPS.
- `CLAUDE.md` holds one line, `@AGENTS.md`; `GEMINI.md` holds `@./AGENTS.md`. Both AIs then read the same rules.

2 · Timeline for today

Time	A · Engine	B · Interface	Together
11:15–11:30	Create repo + skeleton, push `main`	Clone, create branch `interface`, read SPEC.md and CONTRACT.md	Agree the contract and the spec gaps (section 7)
11:30–12:00	Design canvas → send screenshots and values to B; start `mock.js`	`index.html`, `i18n.js`, `ui.js` with plain styling	
12:00–12:30	`recorder.js`, `player.js`, `api.js`, `app.js` against `?mock=1`	`styles.css` from A's design; A− / A+, dark mode, reduced motion	
12:30	**MERGE 1** – the full flow runs with `?mock=1`. Show it to the Caritas team over lunch.		
13:00–14:00	Connect the real FastAPI backend; Safari test; barge-in	Keyboard-only and screen-reader pass; 360 px check	
14:00	**MERGE 2** – real backend in, then run acceptance criteria 1–8 together and fix what fails		
14:30	**MERGE 3** – works end to end		
14:30–15:30	README; record a 90-second backup video of the best run	Copy check in both languages; final polish	Help with the pitch
15:30	**CODE FREEZE** – final merge, tag `v1.0`; only rehearsal after this. Pitches start 16:30.		

3 · The contract between A and B (copy into CONTRACT.md)

```
// js/ui.js — owned by B. DOM only: no fetch, no audio, no session logic.
init(handlers)               // { talkDown, talkUp, talkToggle, sendText(text), replay(url),
                             //   stop, startOver, retry, languageChanged(lang) }
setState(state, detail)      // 'idle' | 'listening' | 'thinking' | 'speaking' | 'error'
                             // detail: { seconds, level, error }
addUser(text)                // user bubble with the transcript
addAssistant(text, audioUrl) // assistant bubble with a replay button
addService(service)          // highlighted card; earlier cards stay; unknown fields listed
setBackendOk(ok)             // show a banner when GET /health fails
clear()                      // Start over
languagePref()               // 'auto' | 'de' | 'en'   (from js/i18n.js)
```

Error codes B needs a DE and EN message for. From the backend: `empty_audio`, `no_speech`, `server_busy`, `internal`. From A's code: `mic_denied`, `mic_unavailable`, `network`, `timeout`.

Backend API stays as in spec §6: `POST /talk` (multipart: audio or text, language, session_id) returns transcript, detected_language, reply_text, reply_audio_url, needs_more_info and service; plus `GET /audio/{id}.wav` and `GET /health`.

4 · Acceptance criteria and who leads them (spec §10)

#	Criterion	Lead	Support
1	A German request gets a spoken and written follow-up or recommendation	A	B (bubbles)
2	English works too; the DE/EN toggle changes all UI text at once	B	A (language field)
3	Service card shows name, reason and at least one working contact link	B	–
4	The second turn is sent with the same session_id and uses prior context	A	–
5	Every error state shows a friendly message; recovery without reloading	A	B (messages)
6	Works keyboard-only, and a screen reader announces replies	B	A (Space / Esc)
7	Notice banner (no counselling, emergency numbers) visible in every state	B	–
8	No horizontal scroll at 360 px; no console errors during a full run	B	A (console)

5 · How we merge

Each of us works on our own branch: A on `engine`, B on `interface`. Because no file has two owners, merges should be conflict-free. A does the merge at every merge point while B watches.

Once, at 11:15

```
# A: create the GitHub repo "service-navigator" and add B as a collaborator
git clone <repo-url> && cd service-navigator
#    add SPEC.md AGENTS.md CLAUDE.md GEMINI.md CONTRACT.md README.md + empty files
git add -A && git commit -m "skeleton" && git push origin main
git checkout -b engine

# B:
git clone <repo-url> && cd service-navigator && git checkout -b interface
```

Every 20–30 minutes, both of us

```
git add -A && git commit -m "ui: service card renders"    # small, says what works
git push origin interface                                 # A: git push origin engine
git fetch origin && git merge origin/main                 # before every new AI task
```

At each merge point (12:30, 14:00, 14:30, 15:30)

```
# 1. Both push their branch first. Then A runs:
git checkout main && git pull origin main
git merge origin/interface
git merge engine
python3 -m http.server 5500      # open http://localhost:5500/?mock=1 and run the smoke test
git push origin main
git checkout engine && git merge main

# 2. B then runs:
git fetch origin && git merge origin/main
```

- **If something conflicts:** it can only be a shared file (`AGENTS.md`, `CONTRACT.md`, `README.md`); A resolves it. If an AI edited a file its person doesn't own, throw that change away: `git checkout origin/main -- <file>`.
- **No GitHub?** B sends her four files (`index.html`, `styles.css`, `js/i18n.js`, `js/ui.js`) through a shared folder and A copies them in. Since no file is shared, copying is merging.

6 · Smoke test after every merge (2 minutes)

- ☐ Page loads with no console errors; the notice banner is visible
- ☐ Talk → listening → thinking → reply bubble appears and is spoken
- ☐ Answering a follow-up sends the same session_id
- ☐ Service card shows; the tel: and mailto: links work
- ☐ `?mock=1&error=server_busy` shows a friendly message and Retry works
- ☐ The DE/EN toggle changes every text at once
- ☐ Keyboard: Tab to the talk button, Space records, Esc stops playback
- ☐ At 360 px wide there is no horizontal scroll

7 · Settle these spec gaps in the first 10 minutes

1. The thinking label “Ich höre zu…” means “I'm listening”. Use it for the listening state; thinking becomes “Einen Moment…” / “Thinking…”.
2. Safari's fallback recording is usually audio/mp4, but the contract lists only webm/opus or wav. Ask the backend to accept mp4 as well, for example by decoding with ffmpeg.
3. If `reply_audio_url` is empty because speech generation failed, show the text, skip the audio, and don't enter the error state.
4. If nobody touched the DE/EN toggle, follow `detected_language` (the spec says the toggle wins over detection).
5. Notice banner: ask Caritas whether to add Telefonseelsorge 0800 111 0 222 and Krisendienste Bayern 0800 655 3000 (24/7).

8 · Starter prompts (paste as the first message to each AI)

A · Claude Code

Read AGENTS.md, CONTRACT.md and SPEC.md (sections 5, 6, 9, 11). Plan first, then build app.js plus js/api.js, js/recorder.js, js/player.js and js/mock.js as vanilla ES modules with no dependencies. app.js runs the section 5 state machine, including hold-to-talk versus tap-to-toggle, barge-in, Esc to stop playback, the 30 s recording cap, the 60 s request timeout and a new session_id on Start over. Talk to the UI only through the ui.js functions in CONTRACT.md. Never edit index.html, styles.css, js/ui.js or js/i18n.js.

B · Gemini

Read AGENTS.md, CONTRACT.md and SPEC.md (sections 4, 7, 8). Build index.html, styles.css, js/i18n.js and js/ui.js in vanilla HTML, CSS and JS: no build step, no CDN, no web fonts, inline SVG icons. ui.js exports exactly the functions in CONTRACT.md and only touches the DOM. Meet section 8: 18 px base text, A−/A+ up to 200 %, 48 px touch targets, a 96 px talk button with icon, label and colour for each state, an aria-live conversation list, aria-pressed with state-based labels, visible focus, light and dark mode, reduced motion, no horizontal scroll at 360 px. Never edit app.js or the other files in js/.